

Digital Systems Design Lab

(BECE102P)

Dr. Sukanta Kumar Tulo
SENSE, VIT Vellore

Course Objectives

To apply theoretical knowledge gained in the theory course and get hands-on experience of the topics.

Course Outcome

- 1. Design, simulate and synthesize combinational logic circuits, data path circuits and sequential logic circuits using Verilog HDL.**
- 2. Design and implement FSM on FPGA.**
- 3. Design and implement small digital systems on FPGA.**

Indicative Experiments

- ADDER & SUBTRACTOR (GATE LEVEL)
- MUX/DEMUX/DECODER/ENCODER/ CODE CONVERTERS/
DESIGN PROBLEMS (GATE LEVEL)
- ADDER & SUBTRACTOR (DATAFLOW LEVEL)
- MUX/DEMUX/DECODER/ENCODER (DATAFLOW LEVEL)
- MULTIPLIER DESIGNS WITH VERILOG HDL (GATE
LEVEL/DATAFLOW)
- FF AND CONVERSIONS
- SHIFT REGISTERS, COUNTERS
- FSM

Verilog Module

- In verilog, the basic unit of hardware is called **module**.
- Modules **cannot** contain the definitions of other module
- A Module can, however, be **instantiated** with in another module.
- Allows the creation of a hierarchy in a Verilog description.
- **Semicolon ;** Statement terminator
- **//** - Single line comment
- **/* */** - Multi-line comment
- Case sensitive

Module Structure

```
module name(portlist);  
    port declarations;  
    parameter declarations;
```

```
    wire declarations;  
    reg declarations;  
    variable declarations;
```

```
    module instantiations;  
    dataflow statements;  
    always blocks;  
    initial blocks;
```

```
    tasks and functions;
```

```
endmodule
```

Ports

Three types of ports

- Input (keyword - **input**)
- Output (keyword – **output**)
- Inout (keyword – **inout**)

Port declarations example

input a;

input a, b;

input [3:0] c, d;

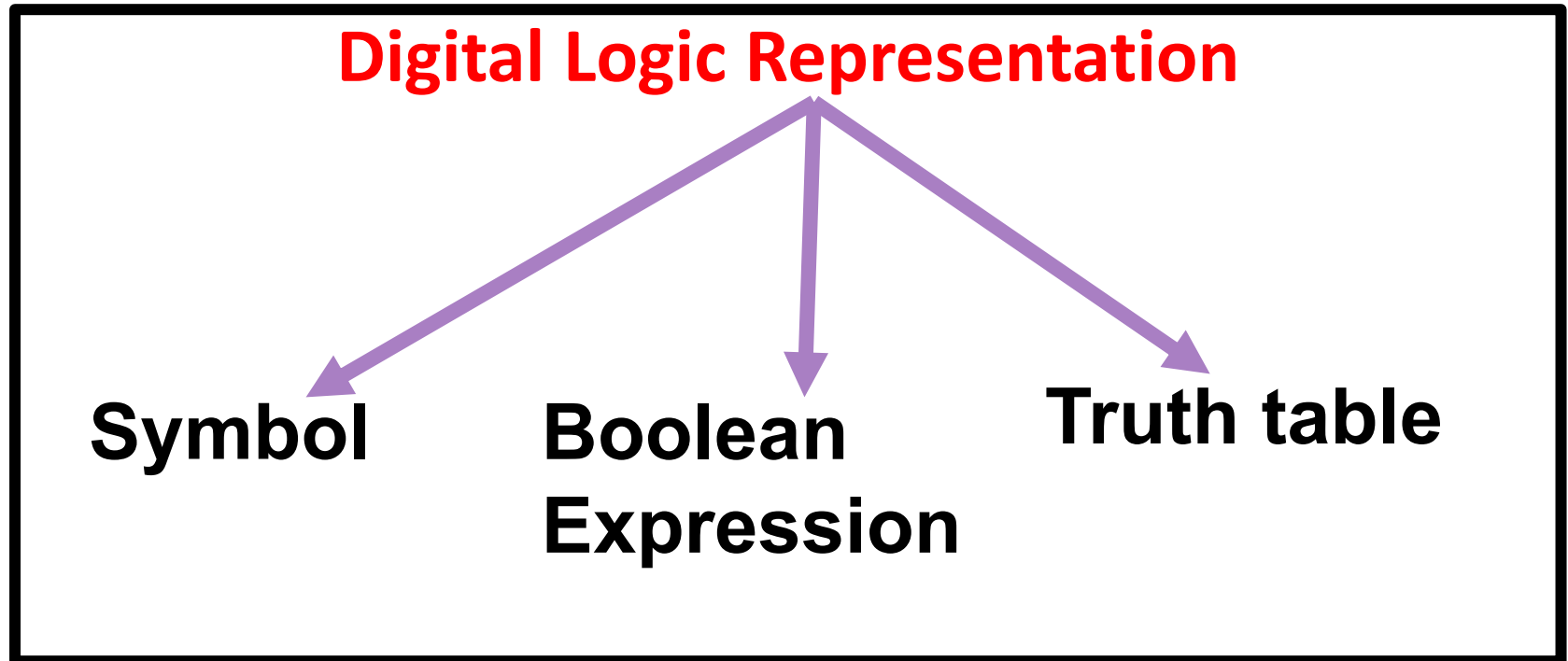
output [4:0] y;

inout x;

Example: XOR Gate

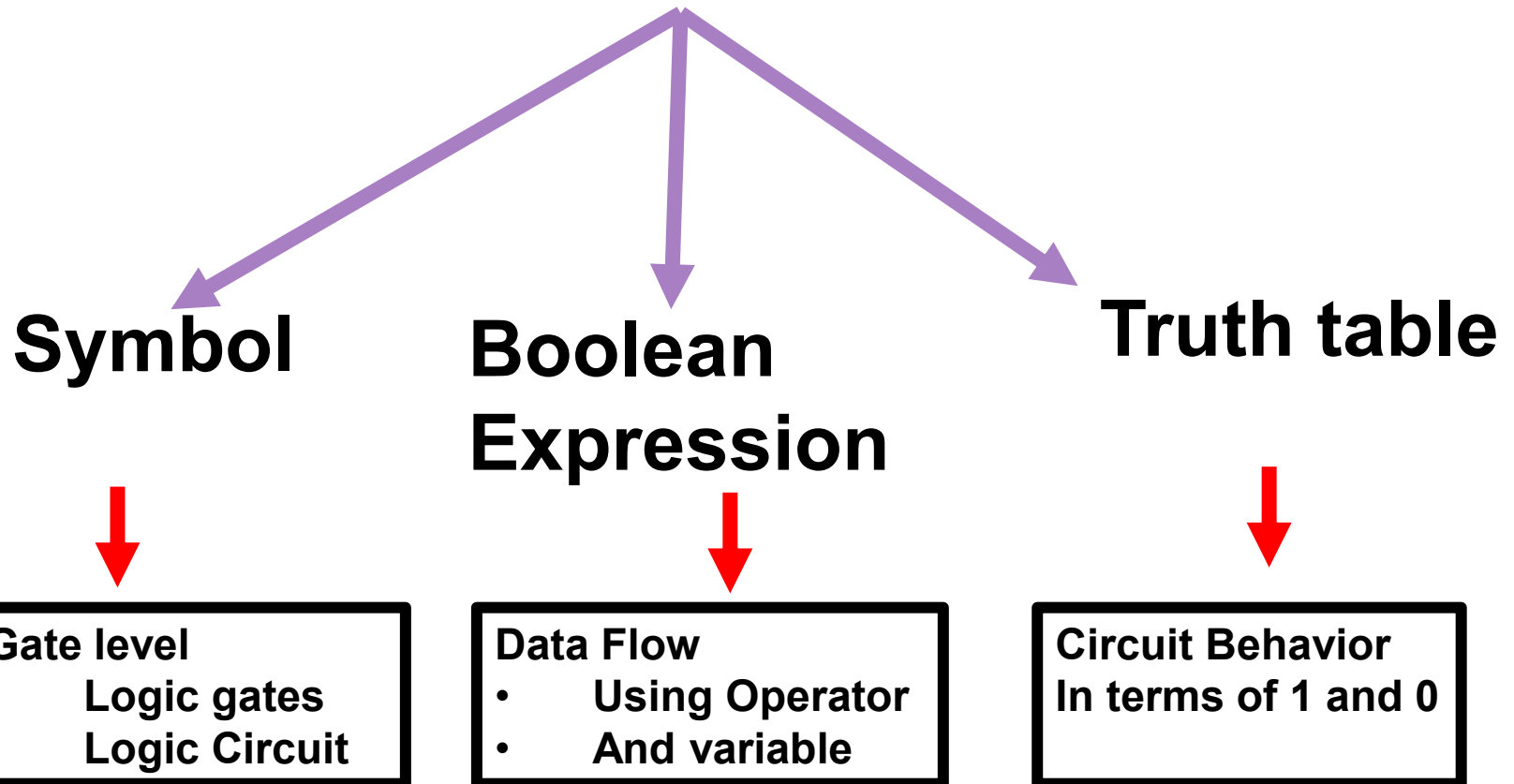
```
module simplexor (f, a, b);  
    input a, b;  
    output f;  
    assign f = a^b;  
endmodule
```

Introduction to Verilog



Introduction to Verilog

Digital Logic Representation



Gate Level/ Structural Modeling

- A Gate level description is based on the structure of the logic circuit.
- A gate level description uses the verilog primitive gates to produce the desired output

Verilog Primitive Gates

- `and(f,a,b,...)` $f = (a \cdot b \dots)$
- `or(f,a,b,...)` $f = (a + b + \dots)$
- `not(f,a)` $f = a'$
- `nand(f,a,b,...)` $f = (a \cdot b \dots)'$
- `nor(f,a,b,...)` $f = (a + b + \dots)'$
- `xor(f,a,b,...)` $f = (a \oplus b \oplus \dots)$
- `xnor(f,a,b,...)` $f = (a \odot b \odot \dots)$

- You can also “name” the gates if you like ...
 - `and And1(z1,x,y);` // this gate is named And1

Dataflow Modeling

- A dataflow description is based on function rather than structure.
- A dataflow uses a number of operators that act on operands to produce the desired function
- **Boolean equations** are used in place of logic schematics.
- **Continuous assignment(assign)** is the most basic statement used in dataflow modeling

Operators

Types of Operators	Symbols
Arithmetic Operators	+, -, *, /, %
Relational Operators	<, <=, >, >=
Logical Equality Operators	==, !=
Case Equality Operators	===, !==
Logical Operators	!, &&,
Bit-wise Operators	~, &, , ^, ~^
Unary Reduction Operators	&, ~&, , ~ , ^, ~^
Shift Operators	<<, >>
Conditional Operators	? :
Concatenation Operators	{ }
Replication Operator	{ { } }

Verilog Operators

{ }	concatenation
+ - * / **	arithmetic
%	modulus
> >= < <=	relational
!	logical NOT
&&	logical AND
	logical OR
==	logical equality
!=	logical inequality
===	case equality
!==	case inequality
? :	conditional

~	bit-wise NOT
&	bit-wise AND
	bit-wise OR
^	bit-wise XOR
^~ ~^	bit-wise XNOR
&	reduction AND
	reduction OR
~&	reduction NAND
~	reduction NOR
^	reduction XOR
~^ ^~	reduction XNOR
<<	shift left
>>	shift right

Verilog Operators

{ }	concatenation	~	bit-wise NOT
+ - * / **	arithmetic	&	bit-wise AND
%	modulus		bit-wise OR
> >= < <=	relational	^	bit-wise XOR
!	logical NOT	^~ ~^	bit-wise XNOR
&&	logical AND	&	reduction AND
	logical OR		reduction OR
==	logical equality	~&	reduction NAND
!=	logical inequality	~	reduction NOR
===	case equality	^	reduction XOR
!==	case inequality	~^ ^~	reduction XNOR
? :	conditional	<<	shift left
		>>	shift right

AND Gate

```
module and_gate_level(y,a,b);  
input a,b;  
output y;  
and a1(y,a,b);  
endmodule
```

```
module and_gate_level_tb; AND Gate Test Bench
reg a,b; // inputs
wire y; // outputs
and_gate_level a1(.y(y), .a(a), .b(b));
initial
begin
a=0; b=0;
#100 a=0; b=1;
#100 a=1; b=0;
#100 a=1; b=1;
end
endmodule
```

OR Gate

```
module or_gate_level(y,a,b);  
input a,b;  
output y;  
or a1(y,a,b);  
endmodule
```

OR Gate Test Bench

```
module or_gate_level_tb;  
reg a,b; // inputs  
wire y; // outputs  
or_gate_level a1(.y(y), .a(a), .b(b));  
initial  
begin  
a=0; b=0;  
#100 a=0; b=1;  
#100 a=1; b=0;  
#100 a=1; b=1;  
end  
endmodule
```

XOR Gate

```
module xor_gate_level(y,a,b);  
input a,b;  
output y;  
xor x1(y,a,b);  
endmodule
```

Dataflow Modeling

assign y = a & b;

assign y = a | b;

assign y = a ^ b;

assign y = ~(a & b);

assign y = ~(a | b);

assign y = a ~^ b;

assign y = ~a;

Gate Level Modeling – Half Adder

//Half Adder : Structural Verilog Description

```
module ha_gate_level (x,y,s,c);
```

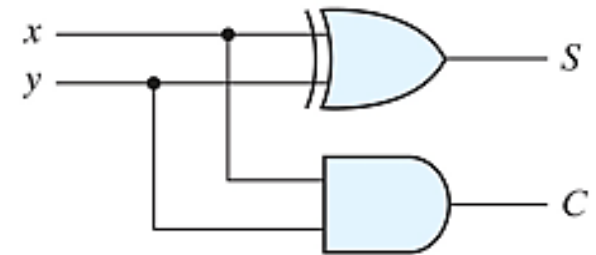
```
  input x,y;
```

```
  output s,c;
```

```
  xor x1(s, x, y);
```

```
  and a1(c, x, y);
```

```
endmodule
```



$$S = x \oplus y$$
$$C = xy$$

Dataflow Modeling – Half Adder

```
//Half Adder : Dataflow description
module ha_df (x,y,s,c);
  input x,y;
  output s,c;

  assign s = x^y;
  assign c = x & y;

  //assign {c,s} = x + y; //Alternate Method

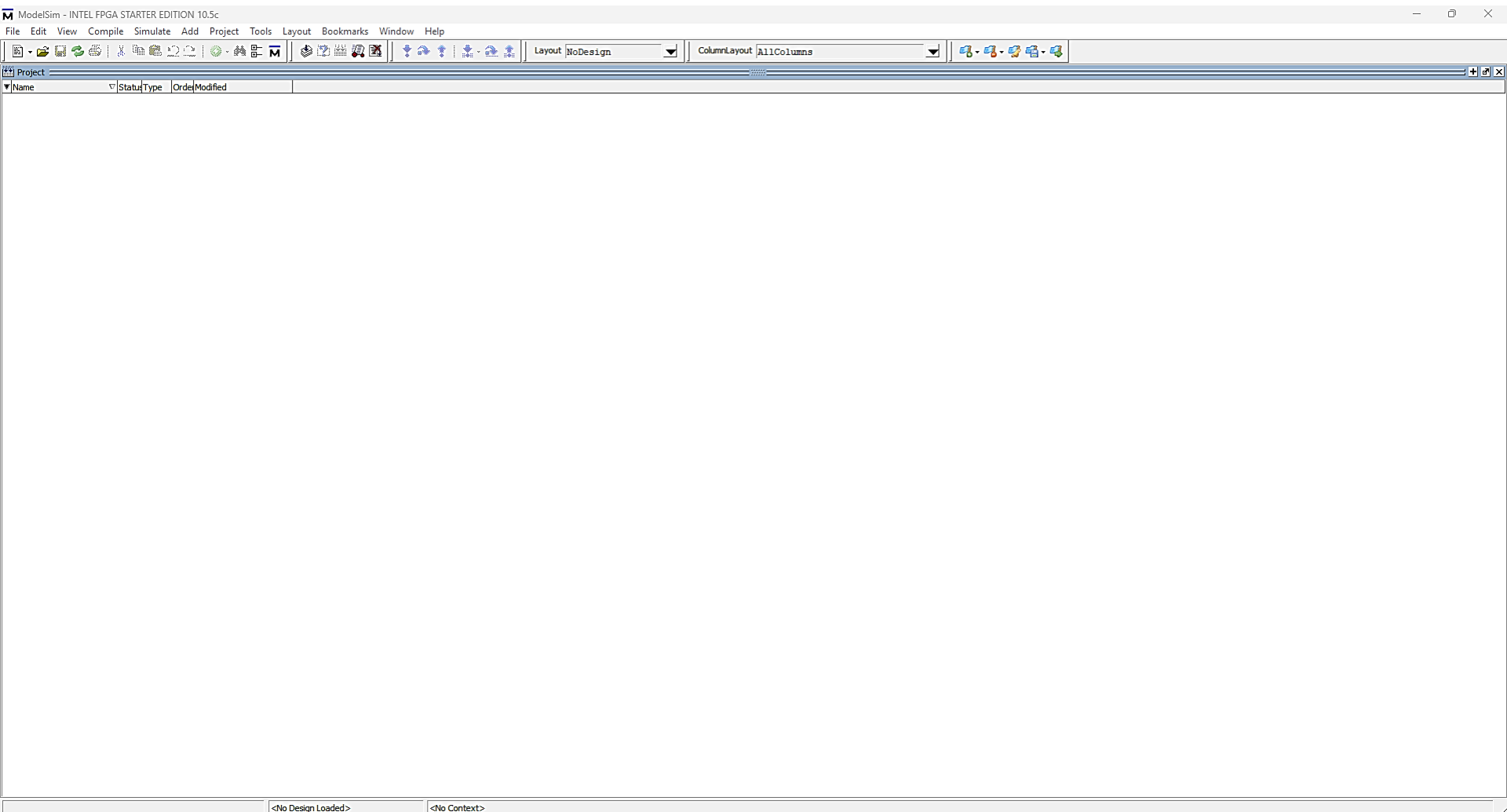
endmodule
```

Modelsim

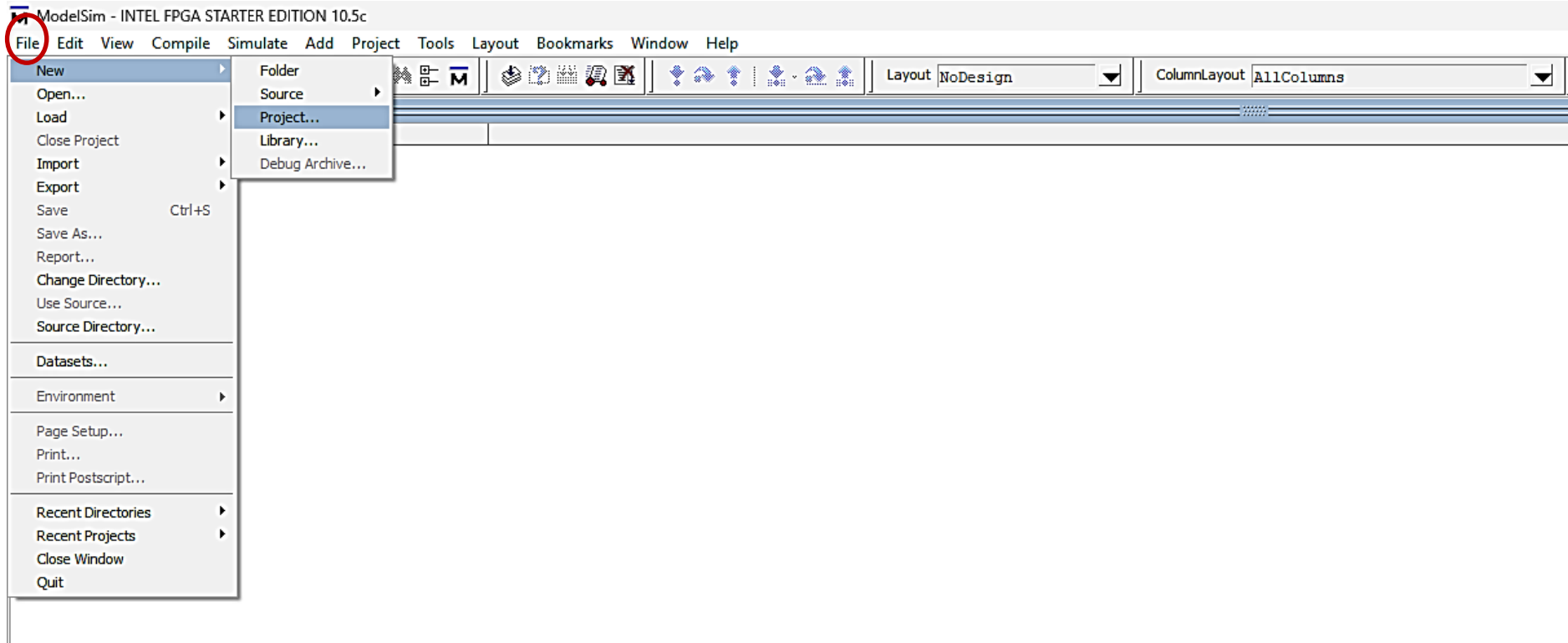
- ModelSim is a simulation verification and simulation tool by Siemens (previously developed by Mentor Graphics) for designs developed using the hardware description languages such as VHDL, Verilog, System Verilog
- ModelSim can be used independently, or in conjunction with Intel Quartus Prime.
- Simulation is performed using the graphical user interface (GUI), or automatically using scripts

Create Folder on Desktop:
Regd_Number_One_Name
21235_Sukanta

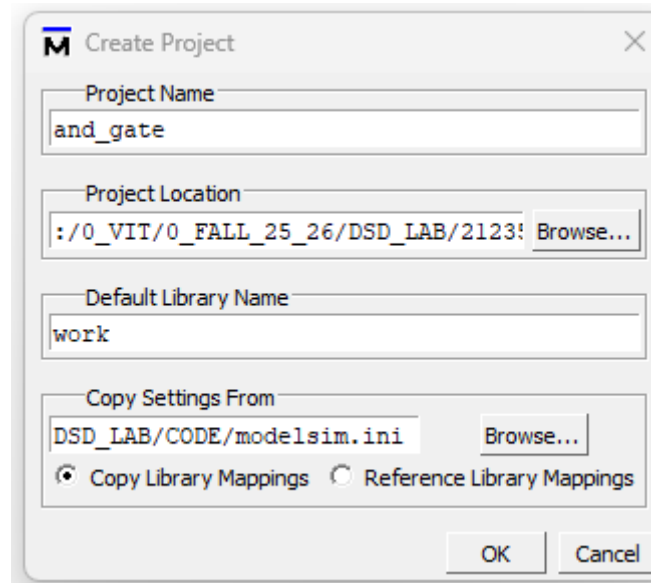
Open ModelSim



Create New Project



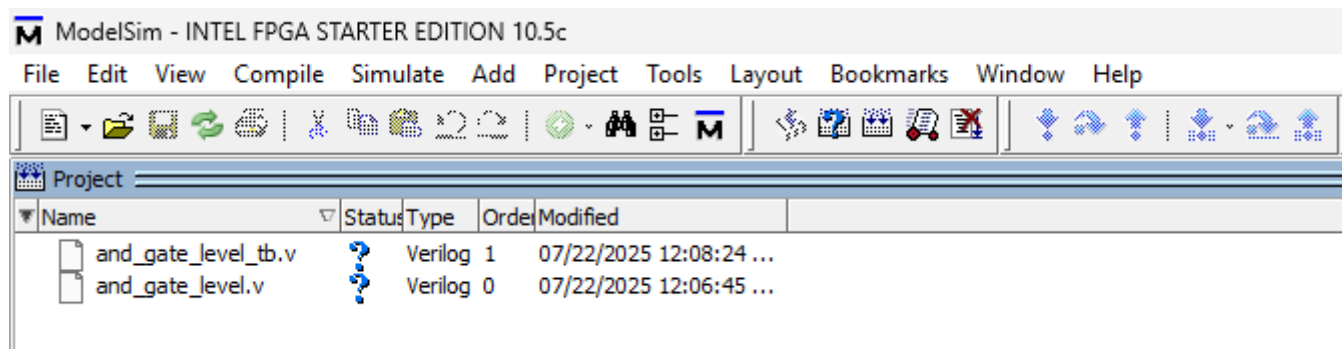
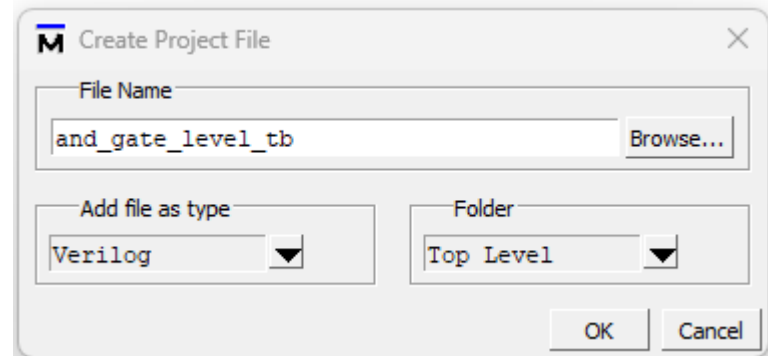
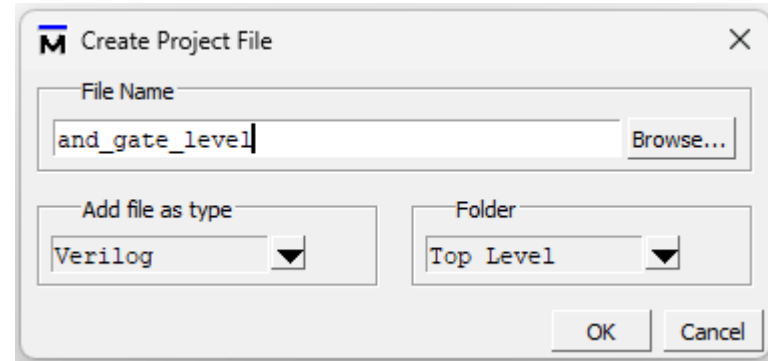
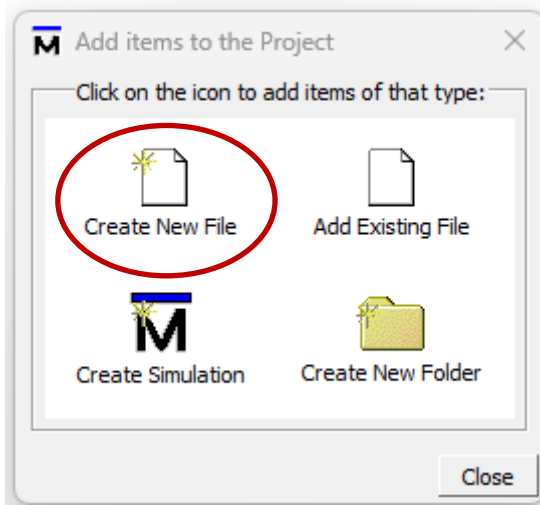
Create Project



The image shows a 'Create Project' dialog box with the following fields and options:

- Project Name:** and_gate
- Project Location:** :/0_VIT/0_FALL_25_26/DSD_LAB/2123: (with a 'Browse...' button)
- Default Library Name:** work
- Copy Settings From:** DSD_LAB/CODE/modelsim.ini (with a 'Browse...' button)
- Radio Buttons:** ☒ Copy Library Mappings, ☐ Reference Library Mappings
- Buttons:** OK, Cancel

Create New File



Edit and_gate_level.v

- Double Click on and_gate_level.v from project list

D:\0_VIT\0_FALL_25_26\DSD_LAB\21235_Sukanta\and_gate_level.v - Default *	
Ln#	
1	<code>module and_gate_level(y,a,b);</code>
2	<code>input a,b;</code>
3	<code>output y;</code>
4	<code>and a1(y,a,b);</code>
5	<code>endmodule</code>
6	
7	

- Save

Edit and_gate_level_tb.v

- Double Click on and_gate_level_tb.v from project list

D:\0_VIT\0_FALL_25_26\DSD_LAB\21235_Sukanta\and_gate_level_tb.v - Default

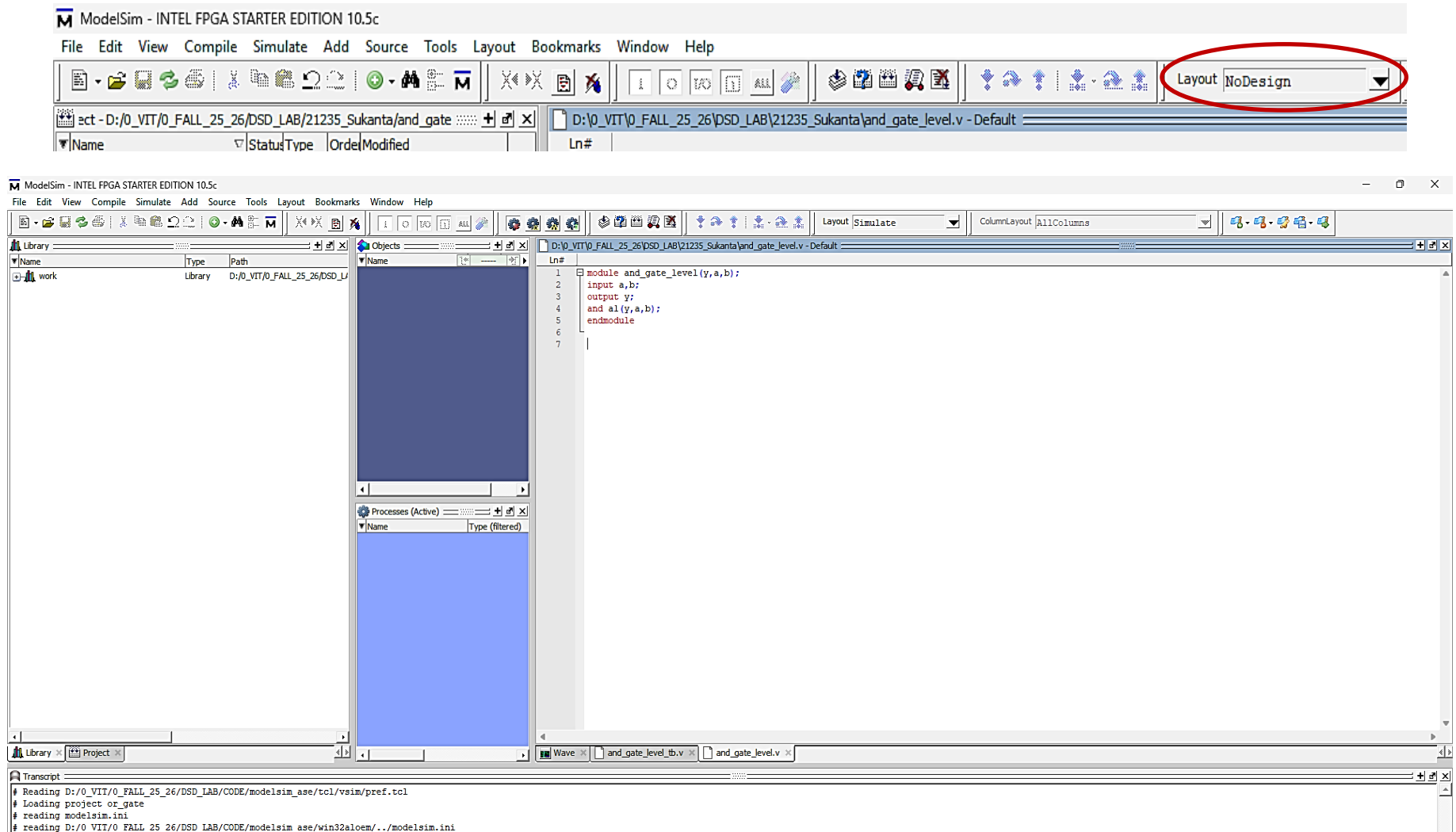
Ln#

```
1  module and_gate_level_tb;
2      reg a,b;    // inputs
3      wire y;     // outputs
4      and_gate_level al(.y(y), .a(a), .b(b));
5      initial
6      begin
7          a=0; b=0;
8          #100 a=0; b=1;
9          #100 a=1; b=0;
10         #100 a=1; b=1;
11     end
12 endmodule
```

- Save

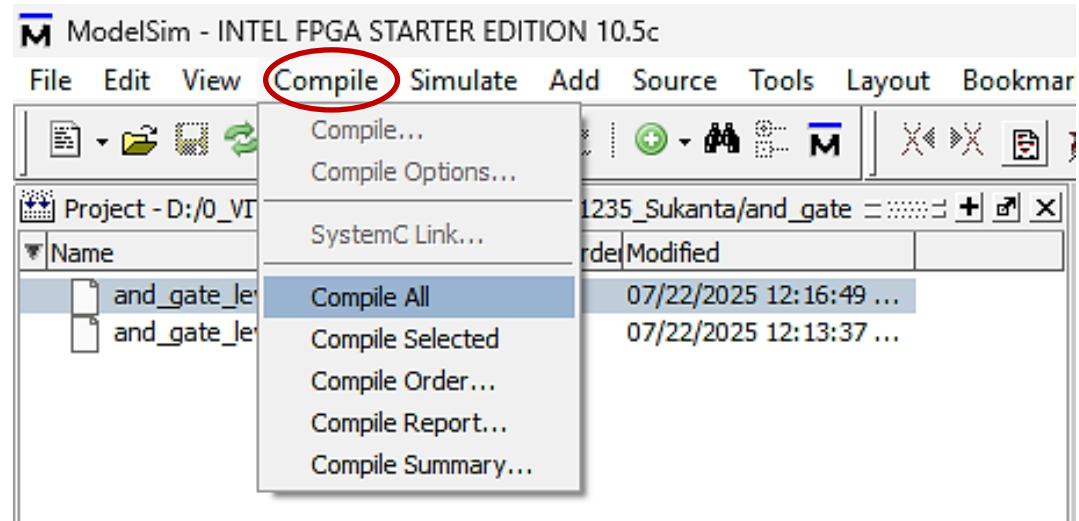
Change Layout to Simulate

- In Layout select Simulate



Compile

- Click on Project Window
- Then, Compile → Compile All

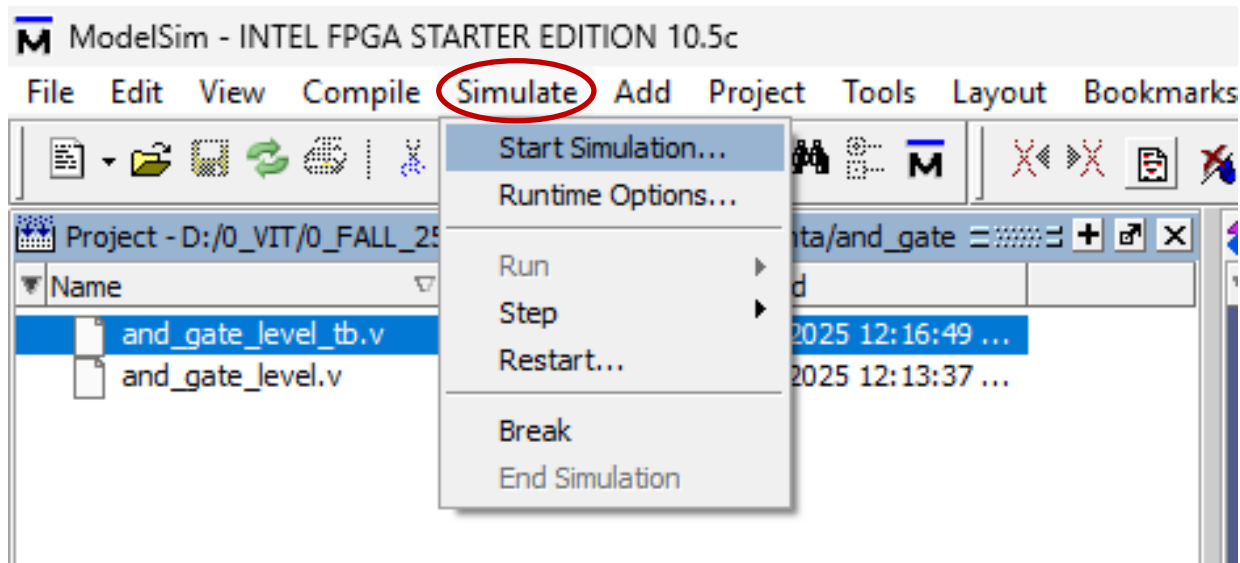


```
Transcript
# Reading D:/0_VIT/0_FALL_25_26/DSD_LAB/CODE/modelsim_ase/tcl/vsim/pref.tcl
# Loading project or_gate
# reading modelsim.ini
# reading D:/0_VIT/0_FALL_25_26/DSD_LAB/CODE/modelsim_ase/win32aloem/./modelsim.ini
# Loading project and_gate
# Compile of and_gate_level.v was successful.
# Compile of and_gate_level_tb.v was successful.
# 2 compiles, 0 failed with no errors.
# Compile of and_gate_level.v was successful.
# Compile of and_gate_level_tb.v was successful.
# 2 compiles, 0 failed with no errors.

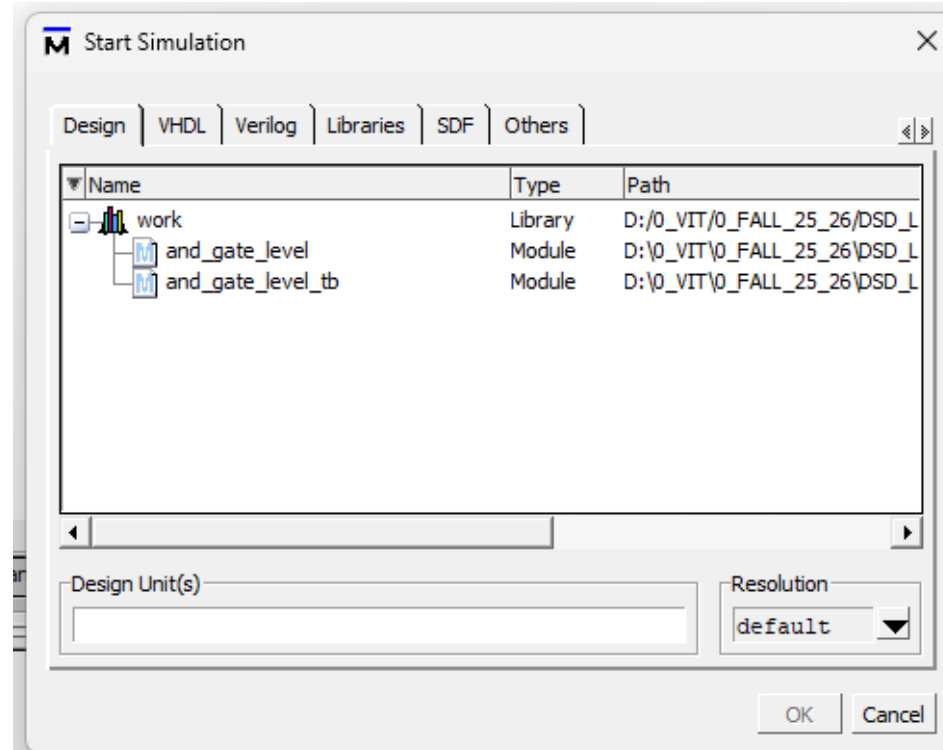
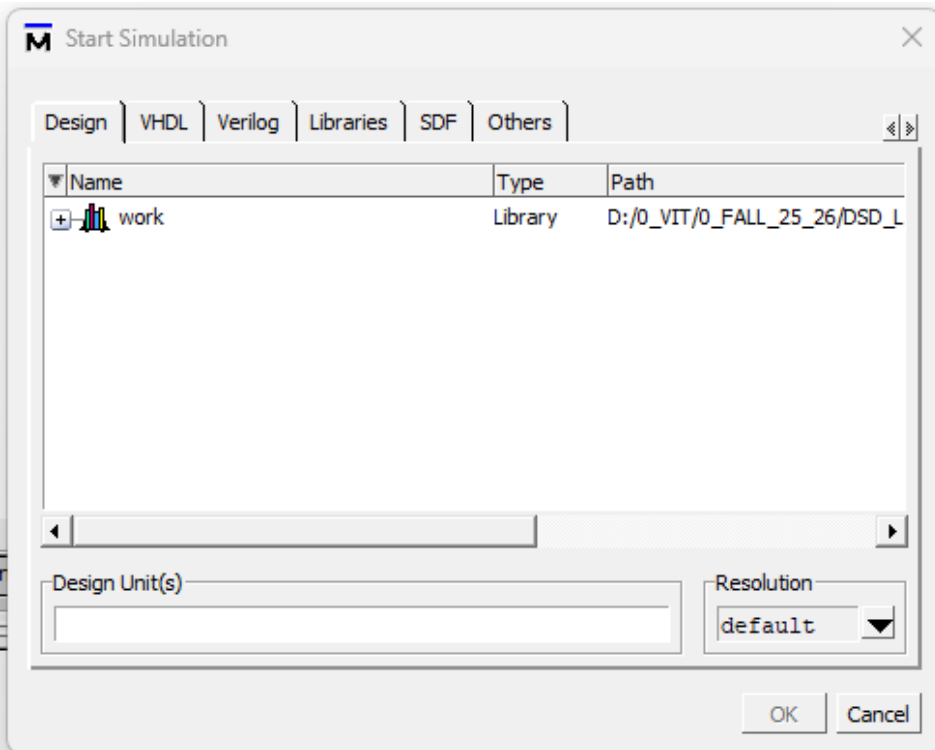
ModelSim>
```

Simulate

- Select and_gate_level_tb.v
- Click Simulate then Start Simulation

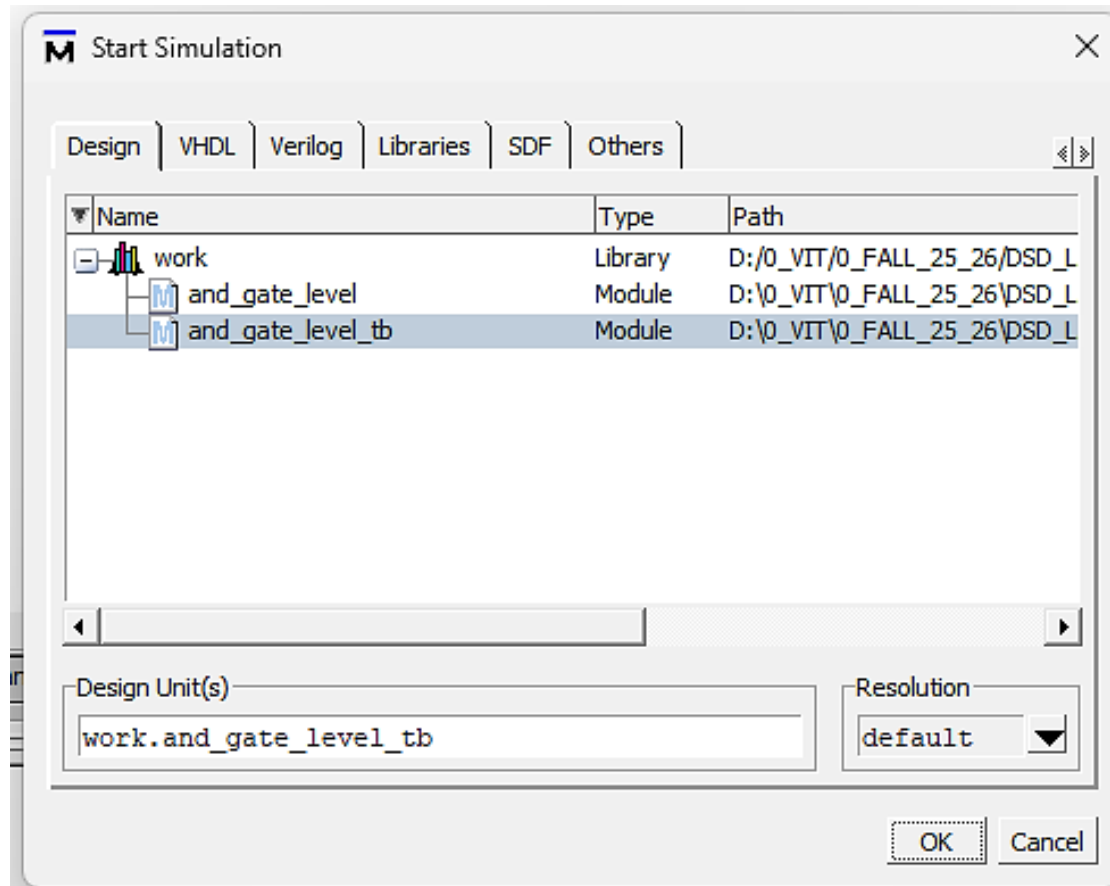


Start Simulation



Start Simulation

- Select and_gate_level_tb and then press ok



ModelSim - INTEL FPGA STARTER EDITION 10.5c

File Edit View Compile Simulate Add Structure Tools Layout Bookmarks Window Help

ColumnLayout AllColumns

sim - Default

Instance	Design unit	Design unit type	Top Category	Visibility
and_gate_level_tb	and_gate_level	Module	DU Instance	+acc=...
a1	and_gate_level	Module	DU Instance	+acc=...
#INITIAL#5	and_gate_level	Process	-	+acc=...
#vsim_capacity#		Capacity	Statistics	+acc=...

Objects

Name	Type	Category
a	x	Regi...
b	x	Regi...
y	StX	Net

Processes (Active)

Name	Type (filtered)
#INITIAL#5	Initial

D:\0_VIT\0_FALL_25_26\PSD_LAB\21235_Sukanta\and_gate_level.v - D

```

1 module and_gate_level(y,a,b);
2   input a,b;
3   output y;
4   and a1(y,a,b);
5 endmodule
6
7

```

Library Project sim

Wave and_gate_level_tb.v and_gate_level.v

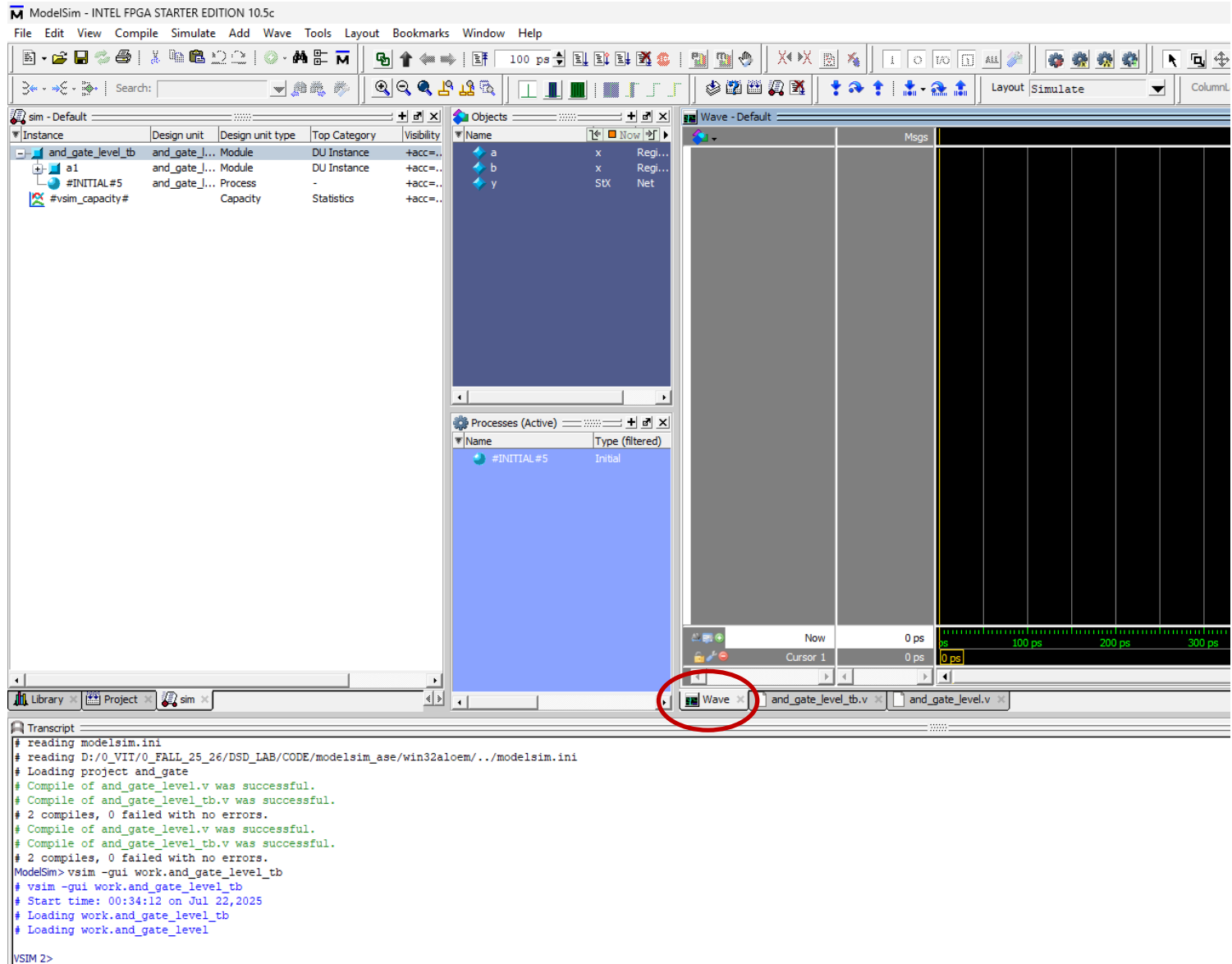
Transcript

```

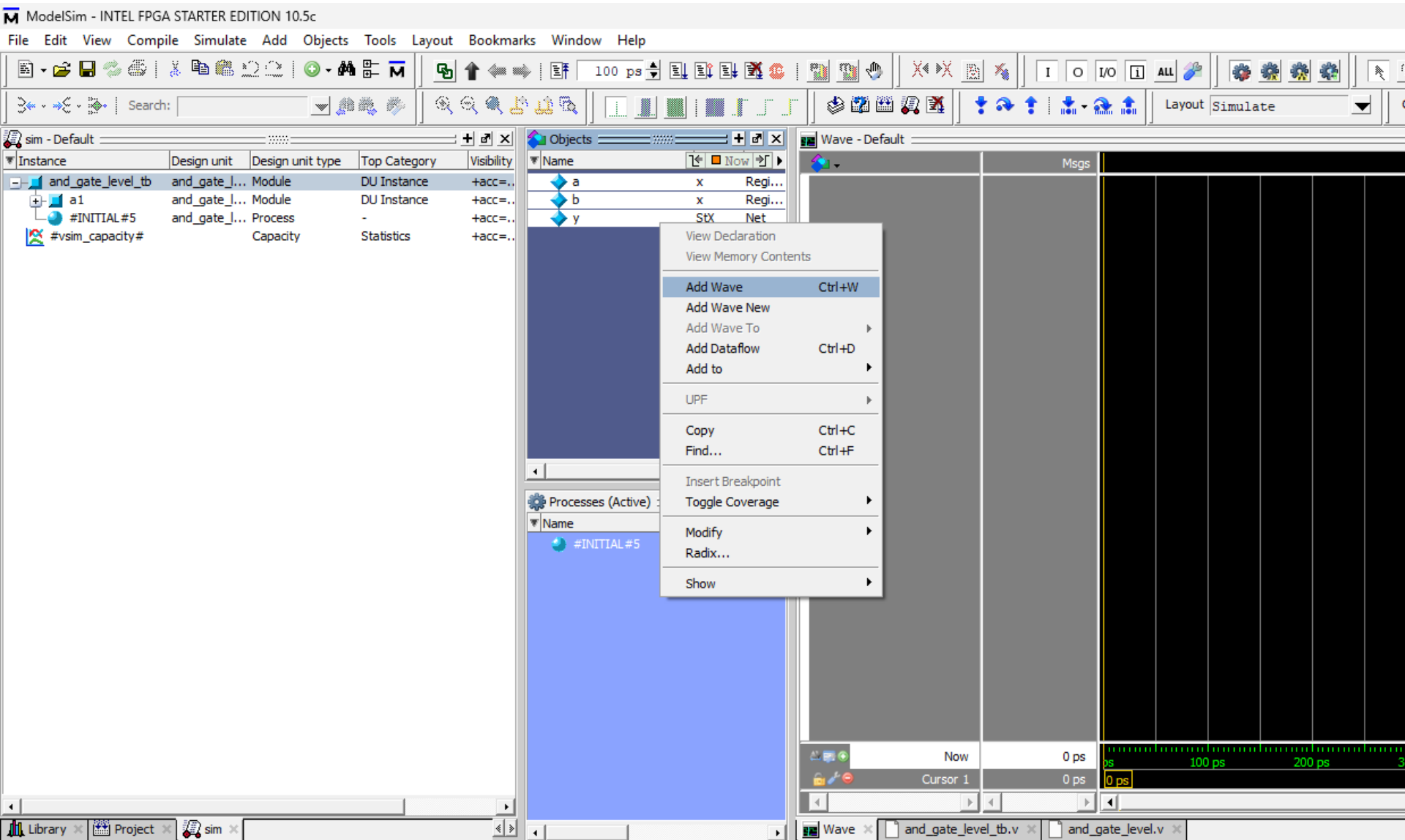
# reading modelsim.ini
# reading D:/0_VIT/0_FALL_25_26/PSD_LAB/CODE/modelsim_ase/win32aloem/./modelsim.ini
# Loading project and_gate
# Compile of and_gate_level.v was successful.
# Compile of and_gate_level_tb.v was successful.
# 2 compiles, 0 failed with no errors.
# Compile of and_gate_level.v was successful.
# Compile of and_gate_level_tb.v was successful.
# 2 compiles, 0 failed with no errors.
ModelSim> vsim -gui work.and_gate_level_tb
# vsim -gui work.and_gate_level_tb
# Start time: 00:34:12 on Jul 22,2025
# Loading work.and_gate_level_tb
# Loading work.and_gate_level

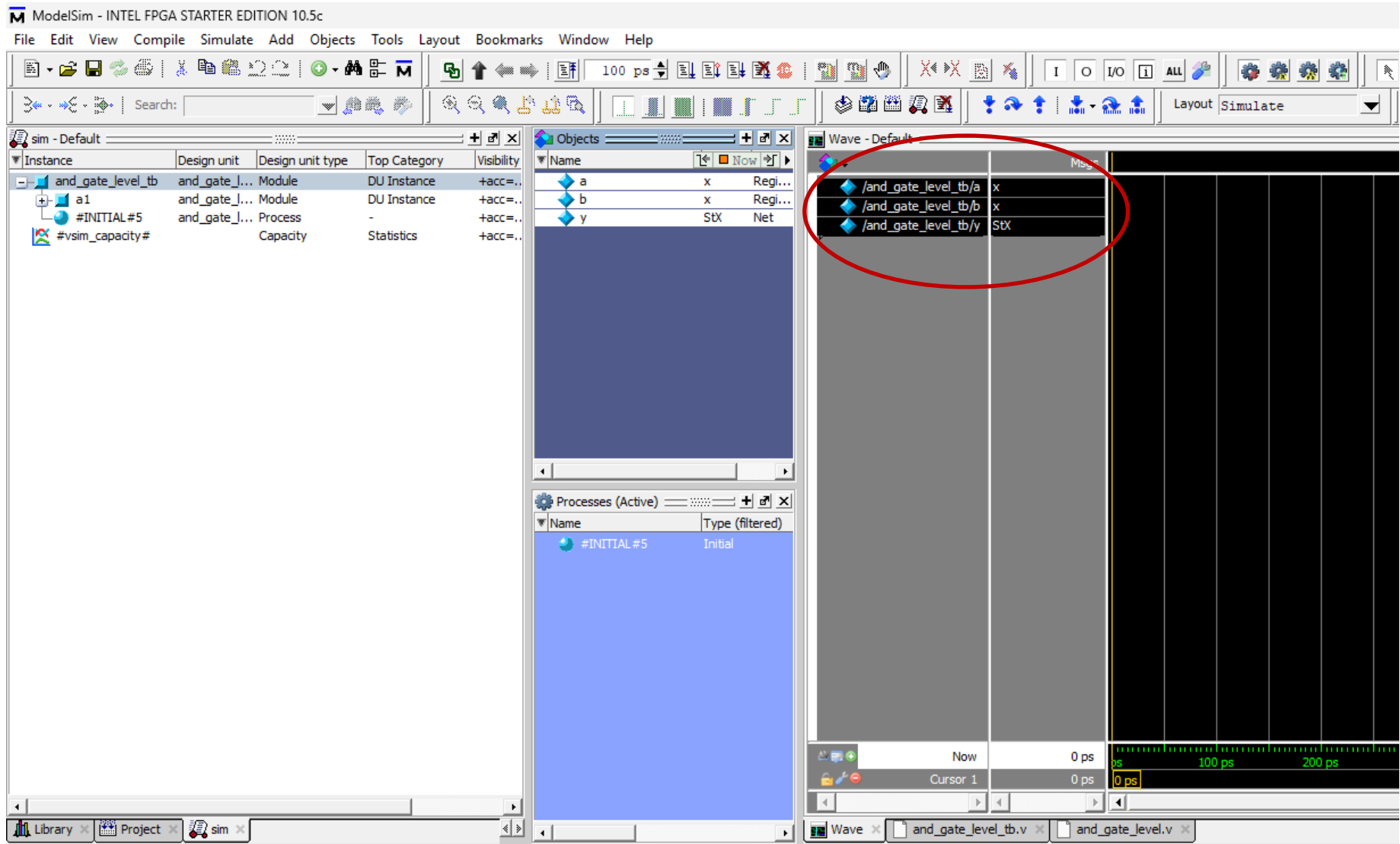
```

- Click Wave window to open it



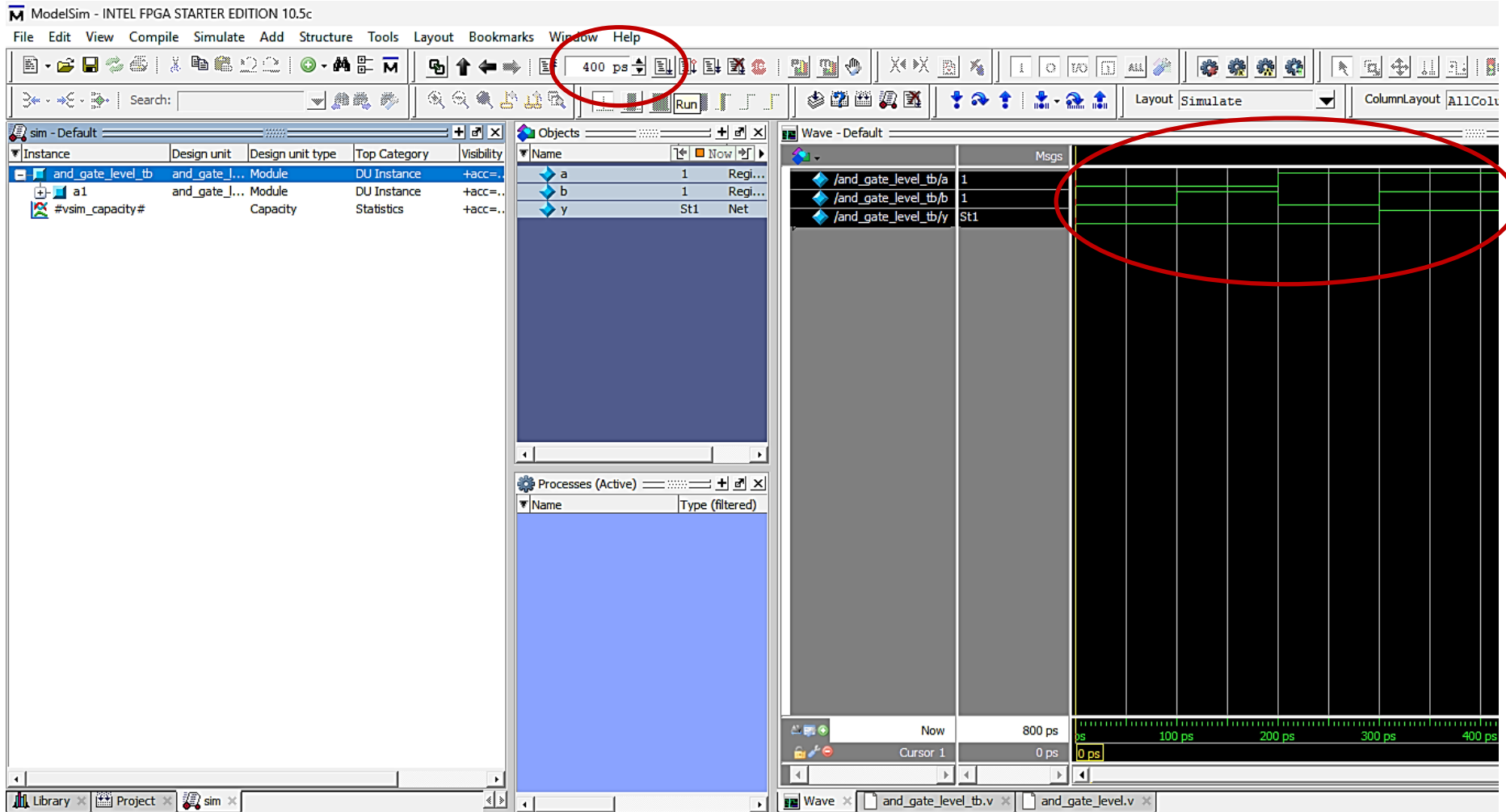
- Select all objects and then right click and then add wave





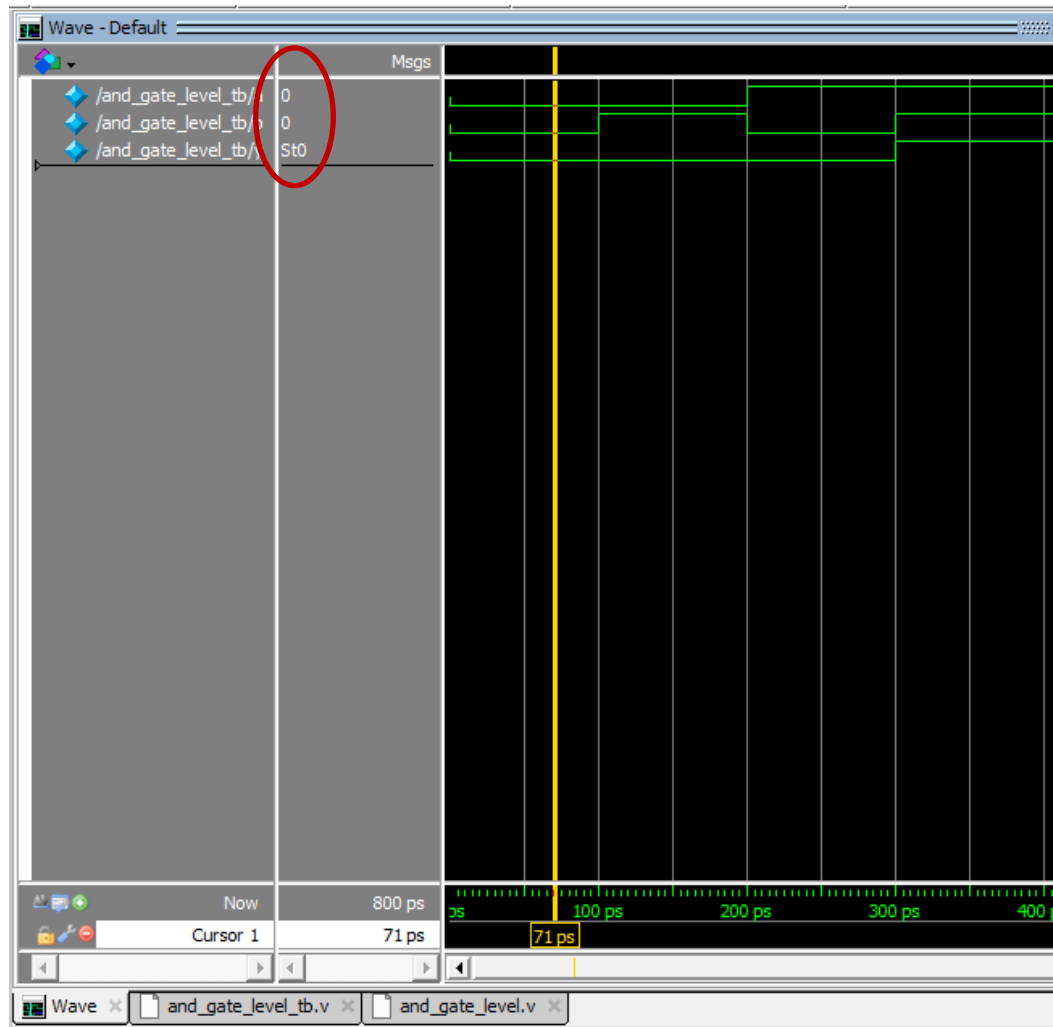
Run

- Change Run Length to 400 ps (As required) and then press Run (present beside that)
- Observe the output waveform



Observe Output

- Click on waveform and then observe



- Save the Screenshot with project name

The screenshot displays the ModelSim GUI with the following components:

- Project - D:\0_VIT\0_FALL_25_26\DSO_LAB\21235_Sukanta\and_gate**: The active project path.
- Files**: A list of files including `and_gate_level_tb.v` and `and_gate_level.v`.
- Objects**: A list of objects including `a`, `b`, and `y`.
- Processes (Active)**: A list of active processes.
- Wave - Default**: A timing diagram showing signals `/and_gate_level_tb/a`, `/and_gate_level_tb/b`, and `/and_gate_level_tb/y`. A red circle highlights the signal `y`.
- Transcript**: A log of simulation commands and results.

The transcript shows the following commands and results:

```
# 2 compiles, 0 failed with no errors.
# Compile of and_gate_level.v was successful.
# Compile of and_gate_level_tb.v was successful.
# 2 compiles, 0 failed with no errors.
ModelSim> vsim -gui work.and_gate_level_tb
# vsim -gui work.and_gate_level_tb
# Start time: 00:34:12 on Jul 22, 2025
# Loading work.and_gate_level_tb
# Loading work.and_gate_level
add wave -position insertpoint \
sim:/and_gate_level_tb/a \
sim:/and_gate_level_tb/b \
sim:/and_gate_level_tb/y
VSI3> run
VSI4>
```

The bottom status bar shows: 0 ps to 1 ns, Project : and_gate, Now: 800 ps, Delta: 0, sim:/and_gate_level_tb.

Thank you